

Names: Dave Burgoin, Moyo Ogunjobi, Jacob Plourde

Group #: 6

Course: ENSF 460 - Embedded Software and Hardware Systems

Assignment Number: Assignment 2 Driver Project

Date Submitted: October 02, 2026

1 Peripheral Initialization

All IO pin initialization is handled in the `I0init()` function, which is called in our `main()` function before the `while(1)` superloop begins.

```
1 void I0init(){  
2     AD1PCFG = 0xFFFF;    // Configure analog-capable pins as digital I/O  
3     TRISB = 0x90;        // Configure RB4 and RB7 as inputs; remaining PORTB pins  
                           // as outputs  
4     TRISA = 0x10;        // Configure RA4 as input; remaining PORTA pins as  
                           // outputs  
5  
6     CNPU1 = CNPU2 = 0x0; // Disable internal pull-up resistors  
7     CNPD1 = 0x3;        // Enable pull-down resistors for RB4 and RA4  
8     CNPD2 = 0x80;        // Enable pull-down resistor for RB7  
9 }
```

2 Superloop

Our `while(1)` superloop singularly contains a call to `I0check()`. `I0check()` is responsible for reading the state of each push-button, and uses a switch statement to determine the output for the user-controlled LED. The structure for this is much like the structure for driver project 1, with the exception that the `delay_ms()` function in this version is implemented with a timer that idles the microcontroller while it is delayed. To ensure that the other LED constantly blinks at 500 ms intervals, it runs on a separate timer, where it is toggled on each timer interrupt.

```
1 #define PB1 ((PORTB & 0x80) >> 7) // Read RB7 into control bit 0  
2 #define PB2 ((PORTB & 0x10) >> 3) // Read RB4 into control bit 1  
3 #define PB3 ((PORTA & 0x10) >> 2) // Read RA4 into control bit 2  
4 #define LED1_MASK 0x0200          // RB9  
5 #define LED2_MASK 0x0040          // RA6  
6 #define LED_TOGGLE_1 (LATB ^= LED1_MASK)  
7 #define LED_ON_1      (LATB |= LED1_MASK)  
8 #define LED_OFF_1     (LATB &= ~LED1_MASK)  
9 #define LED_TOGGLE_2 (LATA ^= LED2_MASK)  
10 #define LED_ON_2      (LATA |= LED2_MASK)  
11 #define LED_OFF_2     (LATA &= ~LED2_MASK)  
12  
13 void I0check(){  
14     uint16_t control_bit = PB3|PB2|PB1;  
15  
16     switch (control_bit)  
17     {  
18         case 1:  
19             delay_ms(250);  
20             break;  
21         case 2: // PB2 only: blink every 1000 ms  
22             delay_ms(1000);
```

```

23     break;
24     case 4:          // PB3 only: blink every 6000 ms
25         delay_ms(6000);
26         break;
27     case 3:          // PB1 + PB2
28     case 5:          // PB1 + PB3
29     case 6:          // PB2 + PB3
30     case 7:          // All three buttons
31         delay_ms(1);
32         break;
33     case 0:          // No buttons pressed
34     default:         // Safe default for any unexpected control value
35         LED_OFF_2;
36         break;
37 }
38 }

```

3 Timer Logic

The timer counts upward at a rate determined by the system clock (250 KHz for an F_{cy} of 125 KHz) and the selected prescaler (1:1 or 1:64). This is the lowest clock speed that we found can be nicely divided for the two different timers. This allows us to save power over the default 4 MHz timer, while not having to change the oscillator speed during runtime.

3.1 delay_ms() Timer Configuration

The code calculates a PR2 value for the requested delay, resets Timer2, and then waits in Idle mode until the timer reaches PR2 and generates an interrupt.

Timer2 was operated with a 1:1 prescaler. With a 500 kHz oscillator, the timer clock is 250 kHz, so the timer increments every 4 microseconds. Since PR2 is a 16-bit register, its maximum value is 65,535. The maximum delay is approximately:

$$\frac{1}{250000} \times 65535 = 0.262 \text{ seconds}$$

So a 0.5 second delay cannot be achieved with the 1:1 prescaler because it would require a PR2 value of 125,000, which is too large for a 16-bit register. A larger prescaler must be used. Our implementation follows:

```

1 // main.c
2 CLKDIVbits.RCDIV = 0b001;
3 __builtin_write_OSCCONH(0b110);
4 __builtin_write_OSCCONL(OSCCON | 0x01);
5 while (OSCCONbits.OSWEN);

```

```

1 // TimeDelay.c
2 void delay_ms(uint16_t delay) {
3     T2CONbits.TON = 0;
4     if (delay > 500){

```

```

5    T2CONbits.TCKPS = 2;
6    PR2 = delay*2;}
7    else{
8        T2CONbits.TCKPS = 0;
9        PR2 = delay*125;}
10   TMR2 = 0;
11   IEC0bits.T2IE = 1;
12   isr_flag = 0;
13   T2CONbits.TON = 1;
14   ...

```

With this configuration, it is possible to give `delay_ms()` a value that is too large. The maximum delay is limited by the 16-bit PR2 register (max 65535) and the largest timer prescaler. With a 1:64 prescaler and a timer clock of 125 kHz:

$$\frac{64}{125000} \times 65535 = 33.55 \text{ seconds}$$

Therefore, the largest delay that can be handled directly is about 33.55 seconds. Anything larger would have to be split into multiple timer periods. if the input to `delay_ms` is less than 500 we use the 1:1 prescaler to get enough resolution for the high frequency cycles.

3.2 Idle() During delay_ms()

While the `delay_ms()` function is blocking until the specified number of milliseconds have passed, it does not need to be wasting clock cycles just iterating a `for` loop, as it was done in driver project 1. This implementation of `delay_ms()` will call `Idle()` after it has configured its timer with the appropriate prescaler and period values, and the microcontroller will free this idle state when the timer interrupt triggers. Because we have two separate timers to control each of the LEDs, this `Idle()` is continuously be called whenever it is awoken until the correct ISR is called and sets the `isr_flag` flag:

```

1  while(!isr_flag){
2      Idle();
3  }

```

This means that during a delay, the CPU will be idle for a significant majority of the time, thereby saving power.